Task 1 — Web App QA & Debug Report

Automation & QA Developer Take-Home Skills Assessment

Candidate: Ajay Kumar
Date: 6/1/2026
Target App: Conduit (RealWorld Demo Application - React-Redux / Express Client)

1. Bug Report & UX Issues Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Sev.	Cause
1	**Broken Profile Avatar URL Validation**	1. Navigate to Settings page (/#/settings). 2. Input an invalid URL (e.g. google.com) in "URL of profile picture". 3. Click Update settings.	Expected: Client/server validation rejects the URL, or falls back to a default avatar image. Actual: App saves the invalid URL and renders a broken image element across all headers.	Low	No URL format validation or image MIME-type check on input. Lack of image onError
2	**Double Form Submission on Slow Networks**	1. Go to Editor page (/#/editor). 2. Fill in fields and click "Publish Article" multiple times rapidly.	Expected: Submit button is disabled on first click and shows a loading state. Actual: Multiple POST requests are sent, creating duplicate articles and throwing database unique constraint errors.	**Medium**	UI component does not track submission state or disable the button during the as
3	**Expired JWT Session Handling (Silent Failures)**	1. Log in and wait for JWT token to expire (or manually modify it in LocalStorage). 2. Attempt to favorite an article.	Expected: App intercepts the 401 Unauthorized API response, logs the user out, and redirects to Login page. Actual: The user remains visually logged in, and interactions fail silently or crash the client state.	**High**	No global HTTP response interceptor to handle 401/403 API errors and trigger a cleanup of
4	**Tag Filter Query Sanitization Failure**	1. Navigate to homepage showing popular tags. 2. Click a tag containing special characters or spaces (e.g., "web dev").	Expected: The tag parameter is URL-encoded before API requests (e.g., %20). Actual: The app appends raw spaces, leading to broken request paths (HTTP 400 Bad Request) from the API.	**Medium**	String concat enation used to build query paths rather than encoding URL components properly
5	**Weak Registration Password Validation**	1. Navigate to Sign Up (/#/register). 2. Input a single-character password (e.g. "1") and submit registration.	Expected: UI enforces password complexity requirements (minimum length, character classes) and password confirmation. Actual: Account is created successfully, introducing serious security vulnerability and typos risk.	**High**	Lack of client-side password confirmation input and weak constraints on the registration API schema.

2. Root-Cause Analysis: Issue #3 — Expired JWT Session Handling

The Conduit React-Redux application stores the JWT authentication token in localStorage and loads it into the Axios headers upon initialization. When the token expires on the backend (or if it is cleared on the server side), any subsequent API request that requires authentication returns a 401 Unauthorized response. However, the frontend application does not listen to or intercept these 401 errors globally. As a result, the application state remains in a logged-in visual state (displaying the user's profile avatar and the logout button), leading to a split-brain UX where the user believes they are authenticated but all operations (such as creating articles, liking posts, or commenting) fail silently or throw unhandled JavaScript errors in the console. To resolve this, a global HTTP response interceptor should be implemented in the API client layer. When a 401 Unauthorized status is detected, the interceptor should automatically dispatch a logout action, clear the invalid token from localStorage and application state, and redirect the user back to the login page with a user-friendly flash message explaining that their session has expired.

Proposed Code-Level Fix (Axios Response Interceptor):

```
// src/agent.js or API client setup
import axios from 'axios';
import store from './store';
import { LOGOUT } from './constants/actionTypes';

const agent = axios.create({
  baseURL: 'https://conduit.productionready.io/api'
});

// Response Interceptor to catch 401 Unauthorized globally
agent.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      // Clear local storage and dispatch logout action
      localStorage.removeItem('jwt');
      store.dispatch({ type: LOGOUT });

      // Redirect to login page and notify user
      window.location.hash = '/login?expired=true';
    }
    return Promise.reject(error);
  }
);
```